2026 首届 openvela AI 硬件开发者大赛·作品提交报告

1、信息表

项目	内容
作品名称	智爱守护——基于 openvela 的多模态非接触式 AI 居家看护终端
队伍名称	代码怎么跑不队
团队分工	张晓慧：板级适配与驱动、硬件联调、系统稳定性；高晓颖：音频链路、AIAgent 与语音交互、异常声音检测；张宇彤：LVGL 界面、联网与手机端联动
选题方向	AI 硬件产品创新+新硬件平台适配

2、摘要

面向独居空巢老人，本作品基于 openvela 与 SF32LB52 实现端云协同看护终端：麦克风常开感知，人声上云完成 ASR、对话与 TTS，非人声由端侧 DSCNN-lite（2372 个 float32，9.5KB）识别跌倒、敲击、尖叫，形成检测—确认—报警—通知闭环。本版加 ASR 并行赛跑与 TCP keep-alive：MiMo 与火山同时识别、先返回者用（12 秒超时）。实测：窗口准确率 80.6%、误报率 6.8%；ASR 文本到手 2~9 秒，短句 1 秒内出声，天气/新闻 15~20 秒；固件 7.13MB，占 Flash 42.5%。断网仍可本地识别与报警。

3、正文

3.1 绪论

3.1.1 项目背景与问题定义

（1）应用场景

本作品面向居家养老场景，目标用户为独居老人与空巢老人，以及不在他们身边的子女。终端需在同一台硬件上承担四类核心能力：语音陪伴（日常聊天、查天气、

问时间）、主动提醒（吃药、喝水）、非接触式异常事件感知（摔倒撞击 / 敲击呼救 / 尖叫）、以及一键报警与远程通知。

考虑到目标用户的生理与认知特点，交互必须做减法：只能依赖「说话」和「几个大按钮」；报警入口必须暴露在首屏；所有可能由误触或误报触发的报警动作，都必须先经过一次用户端确认，再决定是否外发。

（2）具体痛点

- "能听清"在 MCU 上并非理所当然。

·本板音频为半双工（录放共用一条模拟通路），系统内多个 app / 多个线程都会争麦克风。实测最严重的现象是录音跑一阵后永久停摆（read 永远超时），并伴随同一音频 fd 被其他 task group 占用导致的 -EBUSY / ENOTTY。问题本质是多任务场景下这台半双工设备「现在归谁」说不清楚。

- 老人不会、也不该需要操作 App。

传统智能硬件沿用手机 App 交互范式，门槛过高。本作品要求交互限定为「语音＋几个大按钮」，且断网或语音链路失败时必须给出明确反馈，而不是停在「处理中…」的中间态。

- "云"随时可能断。

语音链路每一跳都依赖 HTTPS，任何一环抖动都会直接影响体验。本作品在系统层面采用显式降级：报警响铃、本地声音识别、定时提醒全部本地实现，断网照常工作。

- 不能宣称硬件不具备的能力。

本板没有 IMU 与摄像头，因此放弃视觉识别与姿态跌倒检测，以「声学异常感知」作为现有硬件上唯一可行的非接触式安全方案。

3.1.2 技术难点

（1）半双工音频设备在"多 app+多线程"下的持有者判据与残留清理。

本板音频是半双工、录放共用一条模拟通路，且系统内多个 app、多个线程都会

去开这台设备，而 NuttX 里 fd 属于 task group，跨组去停别人的会话只会打到自己的

文件上。本作品的做法是三条判据：真实通路位；持有者判据；read 返回语义分档，

代码集中在 board/contest_board/src/sf32lb52_audio.c、

sf32lb52_audio_in.c。

（2）NuttX 内核 mutex 持有者跟踪在真机上不可靠。

实测在 ai_audio、报警、询问、推送、异步投递等多条路径上均出现«Assertion failed at /sched/semaphore/sem_post.c:63»（解锁线程不是持锁者）。该断言会直接打死持锁线程，或因持有者字段被破坏导致后继线程永久阻

塞。本版的工程结论是：所有短临界区一律改用二值信号量（无持有者跟踪），不再使用 pthread_mutex / nxmutex 保护界面与报警状态。

（3）内核定时等待与 DMA 中断之间的竞态。.

厂商 HAL 将 DMA 中断设为最高优先级 0，BASEPRI 临界区无法屏蔽。等待对象清理与定时器到点回调交错执行时，会触发 sem_waitirq.c 断言。当前缓解手段包括有界等待、私有看门狗心跳、读停滞重开与 DMA 恢复；根本性修复后的长时间稳定性仍需复测。

（4）端侧小模型要在算力受限的 MCU 上做到「能常开」。

朴素 O(N²) DFT 跑不动，CMSIS-DSP 不能依赖，常规 tflite Micro 难以常驻。本作品手写 512 点 radix-2 FFT 作为 mel 前端，并使用深度可分离卷积小网络（2372 个 float32 权重，约 9.5 KB），BatchNorm 折叠进卷积权重，C 侧无任何框架依赖。

（5）云端配额与响应截断导致「有字无声」。

真机实测 MiMo TTS 曾返回 HTTP 429（quota exhausted），并使 140 字节文本的响应超过 896 KB 缓冲被截断，触发粘性降级至火山引擎；而旧火山资源 ID（豆包 TTS 10029）本账号未开通，返回 403，最终整机无声。本版改用火山「语音合成 2.0」（seed-tts-2.0 + zh_female_vv_uranus_bigtts），并将 TTS 分块上限由 64 字降回 32 字。

3.1.3 创新点

（1）场景创新：非接触式异常事件感知。

区别于依赖摄像头和穿戴设备的传统方案，本作品只用麦克风，就把"声学异常检

测"和"语音交互"合并到同一条音频链路上。通过「人声/非人声」门控进行业务分流，这是本作品和"纯语音助手"的根本分界。

（2）技术创新：端侧小模型从零做到可常开。

自研 mel 前端（含手写 512 点 FFT）+DSCNN-lite 前向+3/4 窗口投票后处理，整

条链路零外部依赖，权重仅 2372 个 float32（≈9.5KB）。

（3）交互创新：先报警、再确认、后通知。

任何危险事件先响铃，再由老人端二次确认；屏幕按钮、语音「没事」、手机 MQTT、超时兜底多条确认路径收口到同一把去重闸，任何一条应答都只报警一次。

（4）工程创新：意图快路。

天气、新闻、自我介绍、常用短句不经过大模型决策链，直接查数据源或念预设回复。真机实测天气/新闻首次出声由约 105 秒降至 15~20 秒，短句低于 1 秒。

（5）交付创新：可复现的交付物。

本将所有底层适配整理为三份权威 working-tree 补丁（nuttx / vendor-sifli / ai_agent）并配套一致性、完备性、凭据扫描脚本；系统文档 6 份，可按 README 从零复现固件。

3.2 系统方案设计

3.2.1 系统总体架构

系统分为三部分：外部器件、端侧处理、云端服务。外部器件为麦克风、触摸屏 / 按键、喇叭、屏幕；端侧处理包含声音分流、本地异常识别、确认与报警、屏幕界面；云端服务包含 ASR、大模型对话与 TTS。 系统总体架构见图 1。

图 1 系统总体架构（端 · 边 · 云）

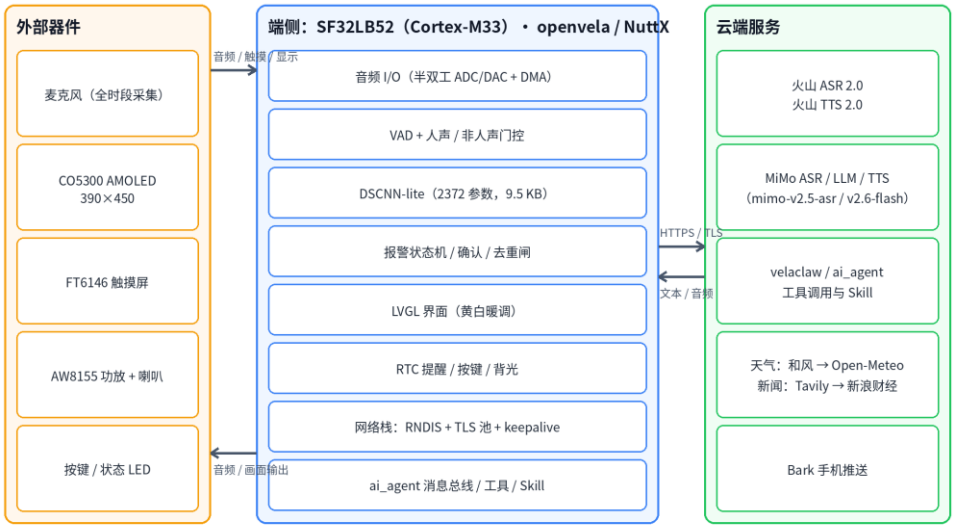


图 1 系统总体架构（端 · 边 · 云）

数据流主要包括四个方面：

（1）麦克风采集声音→声音分流（人声 / 非人声）。

（2）人声 → 云端 ASR → 大模型 → TTS → 喇叭。

（3）异常声 → 端侧小模型 → 确认 → 报警 → 喇叭响铃 + 手机通知。

（4）触摸屏 / 按键 → 屏幕界面 → 确认 / 报警显示。

上电顺序（board/contest_board/src/sifli_ap.c）：robot_ui（优先级 110）→ ai_agent（100）→ hello_app（100）。hello_app 依赖 ai_agent 的消息总线，顺序不可调换。

3.2.2 方案论证与选型

（1）开发板选型理由

本开发板为 openvela 赛事指定平台之一，板载面板、触摸屏、麦克风、功放及原生 USB 接口，无需外接模组即可覆盖「图形显示＋AI 处理＋多媒体」三类核心场景；芯片集成 FPU 并配备 8 MB PSRAM，足以支撑 LVGL 图形框架、中文字库及完整语音链路。

（2）端/云职责划分

端侧（完全离线，不依赖任何云服务）：音频 I/O、VAD、人声门控、端侧声音事件识别、界面渲染、提醒调度、报警动作（响铃+报警页）。

- 云侧：ASR、大模型对话、TTS 三类算力密集型任务，以及手机推送与智能设备指令下发。
- 断网/弱网降级（作为显式设计项）：

  报警与提醒先本地执行，云端仅作增强；

  语音链路失败时弹错误页，不停留在「处理中…」。

（3）与备选方案的对比

备选	本作品的选择与理由
端侧声音分类采用 PyTorch/ONNX 大模型（数 MB 权重+tflite）	不采用。MCU 上无法常开；改用 2372 参数的 DSCNN-lite ＋ 手写 FFT，整条链路零依赖
唤醒采用云端在线唤醒	不采用。本版已取消唤醒词，采用 VAD 常听；交互只依赖「说话」，降低门槛
报警：检测到报警事件即直接上报	采用“二次确认+三条回答入口+去重闸”流程，误报不会直接变成一次报警
板子联网依赖板载 WiFi	采用 USBRNDIS+PC 侧 WinNat 方案（板载 WiFi 模块未接入构建）。

（4）关键模块设计

通信模块：板端以 USB RNDIS（584e:5342）接入 PC 侧 NAT，TLS 连接池统一管理 HTTPS 长连接并开启 TCP keep-alive；本地 MQTT 主题 zhi_ai/zhi_ai_001/command 负责界面、语音与 ai_agent 之间的状态同步，报警外发走 Bark。

协同模块：系统由 robot_ui（UI）、ai_agent（大模型与工具链）、hello_app（语音与音频）三个 task group 组成，通过消息总线与二值信号量协同；音频为半

双工共享设备，由真实通路位、rx_owner_gen 单读者判据与 write 原子串行化仲裁，任一时刻只有一个 owner。

交互模块：常开麦 + VAD 自动断句，无唤醒词；主屏大按钮、全屏红色报警页与语音「没事」三条确认入口收口到同一去重闸；所有云端失败都有本地提示，不停留在「处理中…」。

3.3 核心算法与技术原理

3.3.1 AI 算法实现：端侧与云端

端侧：纯 C 实现的 DSCNN-lite，不使用 tflite / CMSIS-NN / CMSIS-DSP。输入为 64×40 的 log-mel 窗（0.64 s，200 ms 步长），网络为 3×3 卷积 + 深度可分离卷积 + 全局平均池化 + 全连接，共 2372 个 float32 参数（约 9.5 KB）；float32 前向，BatchNorm 已折叠进卷积权重与偏置，未做整数量化。模型常驻 SRAM，不占 PSRAM；单窗前向由门控线程（低优先级）执行，与录音/播放线程解耦，避免阻塞音频 DMA。

云端：ASR 使用 MiMo mimo-v2.5-asr 与火山 volc.seedasr.sauc.duration 双后端并行（12 s 超时，先返回非空文本者用）；LLM 使用 velaclaw（ai_agent ReAct，mimo-v2.6-flash），超时回落 mimo_chat 直连；TTS 使用火山 seed-tts-2.0 + zh_female_vv_uranus_bigtts，单块失败回落 MiMo mimo-v2.5-tts。三项均为 HTTPS。

接口与鉴权：MiMo 走 api.xiaomimimo.com 的 /v1/chat/completions 等接口，使用 Bearer api_key；火山使用 X-Api-Key（36 位 UUID）与对应资源 ID。所有密钥只保存在 ROMFS 的 board/contest_board/src/etc/assets/agent_config.json（.gitignore 已挡），开机拷入 /data/ai_agent/config/config.json，代码与仓库不含真实密钥。

3.3.2 端侧 AI 算法：本地声音事件识别

- **实现位置**：app/hello_app/sound_event.c/sound_event.h，权重：sound_event_model.h；

项	本作品实现
框架	纯 C 实现，无外部依赖：不使用 tflite/CMSIS-NN/CMSIS-DSP
前端	16kHz 单声道 s16le；帧长 400 样本（25ms）、帧移(跳)160（10ms）、symmetricHann 窗；手写 512 点 radix-2FFT（sound_event.c:199se_fft512）→257 点功率谱→40 个三角 mel（HTK 刻度，f_min0/f_max8000）→log(max(x,1e-10))；窗长 40×64（0.64s），滑窗步长 20 帧（200ms）；整窗按(x-mean)/(std+1e-5)归一化

项	本作品实现
网络	DSCNN-lite：conv3×3(s2,1→16)→depthwise3×3(16)→pointwise1×1(16→32)→depthwise3×3(32)→pointwise1×1(32)→全局平均池化→fc(32→4)，每层后接 ReLU
量化	float32，未做整数量化；BatchNorm 已折叠进卷积权重与偏置
权重体积	2372 个 float32≈9.5KB（sSE_W_LEN2372）
算力/内存	2372 参数 / 约 9.5 KB；64×40 输入窗口缓冲约 10 KB；模型常驻 SRAM，不占 PSRAM；低优先级门控线程每 200 ms 处理一窗，M33 常开。
后处理	3/4 窗口投票（看最近 4 个窗口、同类至少 3 票）+每类置信度阈值 0.55+同类应期。
类别	0=other、1=fall、2=knock、3=scream

本版相控策略（真机数据驱动）：

- 人声窗只允许 scream 类投票；人声后 2.5 秒「余温」同样只允许 scream ——解决「一说话就被判成敲击 / 跌倒」。
- fall 与 knock 统一按「撞击」计数：单次置信度 ≥0.80 立即报，否则 15 秒内第二次撞击才报；单次 0.55~0.80 只记时间。
- 连续超过 6 秒的持续声不报撞击，挡掉风扇、摩擦等持续环境噪声。
- 撞击类不应期 2 秒（10 窗），尖叫类 8 秒（40 窗）。

3.3.3 云端 AI 算法：ASR / 对话 / TTS

环节	实现	真机证据
ASR	火山 2.0（volc.seedasr.sauc.duration），失败回落 MiMo mimo-v2.5-asr	Recognized: 你好你好，给我讲个小故事（infer≈400 ms，e2e≈5.8 s）
LLM	velaclaw（ai_agent ReAct，mimo-v2.6-flash），-110 超时回落 mimo_chat 直连	trace iter=1 latency≈7.8 s llm=ok backend=0
TTS	火山 2.0（seed-tts-2.0 + zh_female_vv_uranus_bigtts），单块失败直连 MiMo	volc_tts synthesized 165852 PCM bytes（14 chunks）
天气	QWeather（配置后优先）→ Open-Meteo	QWeather configured

	（免 key 兜底）	
新闻	Tavily（topic=news）→ 新浪财经（免 key 兜底）	News via Tavily OK: N bytes
推送	Bark（报警通知家属）	[PUSH] OK: HTTP 200

3.3.4 关键机制一：「人声/非人声」二分流门控

- 模块：app/hello_app/ai_sound_detect.c。录音线程 read() 到一帧 → sound_detect_pcm_tap()（非阻塞，仅向有界环形缓冲 memcpy）→ 门控线程（独立、低优先级）每 200 ms 取一个 0.64 s 窗，随后分流：
- 播放态（板子自身喇叭在响）→ 整窗丢弃，不判。
- 像人声 → voice 回调 → 交给云端语音链路。
- 非人声且能量超阈 → sound_event_feed_ex() → 本地小模型。
- 硬要求：环形缓冲满时丢弃最旧数据，录音线程永远不会被门控拖住 —— 麦克风是半双工、hello_app 已常开独占，堵住录音就是「整机聋了」。

3.3.5 关键机制二：检测 → 确认 → 报警 → 通知

- 本地小模型命中或语音关键词命中后，进入统一报警状态机（app/hello_app/voice_emerg.c）：
- 响铃 1.2 秒 / 听 3.6 秒循环；10 秒内说「没事」或点屏幕按钮 → 取消，不通知家人。
- 10 秒无人应答 → Bark 推送家属；之后每 30 秒重发，持续响铃。
- 180 秒到点自动收摊（主循环 100 ms 一拍兜底，工作线程卡住也能收）。
- 「没事」识别采用短句兜底：报警确认期内，ASR 返回文本 ≤15 字且不是急救词，一律按「没事」处理。真机实测 ASR 曾把「没事」听成「冤枉」「好，是又调」，关键词匹配追不上所有变体，而确认期用户的唯一合理回答就是「没事」。

3.3.6 关键机制三：意图快路

为避免所有问题都经过大模型决策链，本版为高频意图增加快路：

快路	触发词	实测延迟
短句	你好 / 谢谢 / 再见 / 好的 / 晚安	< 1 秒
自我介绍	你能做什么 / 你是谁 / 你会什么	< 1 秒
天气	天气 / 气温 / 下雨 / 冷不冷 / 热不热	15~20 秒
新闻	新闻 / 头条 / 时要 / 国际消息	15~20 秒
新闻概括	详细说说第一条 / 概括一下第二条	15~20 秒

快路全部在送大模型之前判定，任一条件不满足即回落到原大模型路径，不改变原有语义。

3.3.7 关键机制四（驱动侧）：音频设备持有者仲裁与自愈

驱动（board/contest_board/src/sf32lb52_audio.c）为每次 read 维护 rx_owner_gen：

Read 武装前登记 rx_owner_gen = 当前 session_gen；只有 owner 才能清除 rx_busy。

新读者遇到 rx_busy 一律返回 -EBUSY 让路；持满 6.5 秒判死并抢占。

等待上界 5 秒由驱动私有看门狗产生，不依赖内核 nxsem_clockwait。

本版修复的核心缺陷：旧判据 `rx_busy && !rx_aborted` 会在 stop 之后放行新读者，新读者进门即 nxsem_reset ＋ wd_cancel，把旧读者那记 5 秒心跳拆掉，旧读者从此无上界 —— 这正是真机「录音线程卡在设备读里 8019 ms / 放弃 join / 会话只涨不跌」的根因。

3.3.8 关键机制五：界面线程稳定性

真机曾多次出现「报警页弹出后按钮点不动、界面冻结」。逐层排查后定位为：pthread_mutex 持有者跟踪不可靠，持锁线程异常时后继线程永久阻塞；voice_emerg_cancel 在 LVGL 线程内调用 audio_play_stop（含 join），播放线程卡住时阻塞界面。

本版的处理：robot_ui 与板级报警内全部互斥锁改为二值信号量；取消报警时改由 detached 线程执行 audio_play_stop，界面线程立即返回；并增加 UI 看门狗，主循环 10 秒无心跳即 dumpstack 打印界面线程调用栈。4

3.3.9 openvela 系统能力的深度运用

本作品同时落地「图形 / AI / 多媒体」三项核心能力，并大量使用 openvela 组件：

图形：LVGL 负责 390×450 主界面、菜单、设置与报警页；CO5300 QSPI 面板驱动、FT6146 触摸输入、ROMFS 中文字库与 UI 看门狗构成完整图形栈。

AI：ai_agent 提供 ReAct 工具链、消息总线与 Skill 运行时；端侧 DSCNN-lite 常开做异常声音识别；云端 MiMo/火山承担 ASR、LLM 与 TTS。

多媒体：NuttX audio lower-half 驱动 AUDCODEC/AUDPRC 与 AW8155 功放，完成 16 kHz 单声道录音/播放、半双工切换与 TTS 逐块播放。

使用的 openvela 组件包括：nuttx/net/tcp（连接池与 keep-alive）、usbdev/rndis（联网）、audio（DMA 与半双工）、RTC/GPIO/PWM（提醒、按键、背光）、ROMFS（配置与字库）、libc、LVGL 与 ai_agent。

对 openvela 的优化与拓展：修复 TCP TIME_WAIT 不释放与僵尸 CLOSED 条目（tcp_timer/tcp_conn）；新增板级 TCP keep-alive 配置；修复 RNDIS TXPKTRDY/FIFO flush、bulk OUT 双提交与 QUERY stall；修正 M33 systick 配置名；在音频驱动增加 rx_owner_gen 单读者判据与 write 原子串行化。改进建议：上游可提供半双工音频标准接口、RNDIS warm reset 说明与 TCP keep-alive 默认配置，减少板级重复适配。

3.4 系统实现

3.4.1 软件/固件架构

模块	文件	行数	职责
语音主控	app/hello_app/ai_companion_main.c	9884	主循环、VAD、ASR→LLM→TTS、快路、看门狗
语音后端	app/hello_app/mimo_voice.c	4999	MiMo / 火山后端、分块、HTTP
音频半双工	app/hello_app/ai_audio.c	3144	录音线程、播放、让路锁
声音门控	app/hello_app/ai_sound_detect.c	2230	人声 / 非人声分流
端侧模型	app/hello_app/sound_event.c	1189	mel＋DSCNN-lite＋投票
语音急救	app/hello_app/voice_emerg.c	1106	报警状态机、响铃、Bark
界面主控	app/robot_ui/main.c	5282	LVGL 主循环、询问页、网络任务
界面组件	app/robot_ui/robot_ui.c	1831	主屏、表情、时钟、报警页
联网与推送	app/robot_ui/network_comm.c	2998	MQTT、Bark、推送队列
音频驱动	board/contest_board/src/sf32lb52_audio.c	4751	DMA、半双工仲裁、自愈

板级报警	board/contest_board/src/sf32lb52_alarm.c	831	报警音（独立 fd）

3.4.2 数据流与关键流程设计（附流程图）

语音问答主流程：

麦克风 → VAD 判到说完 → 快照送 ASR（火山，失败回落 MiMo）→ 轻意图分类（灯控 / 急救词 / 短句 / 天气 / 新闻 / 自介）→ 未命中则送大模型 → 逐块 TTS（火山 2.0，10 字首块，32 字后续块）→ 播放 → 播完还麦。

报警主流程：

本地小模型命中或语音关键词命中 → voice_emerg_trigger → 响铃 1.2 s → 听 3.6 s → 10 秒内「没事」或按钮取消 → 否则 Bark 通知家属 → 每 30 秒重发 → 180 秒自停。 语音问答与报警流程见图 2。

图 2 语音问答与异常报警流程

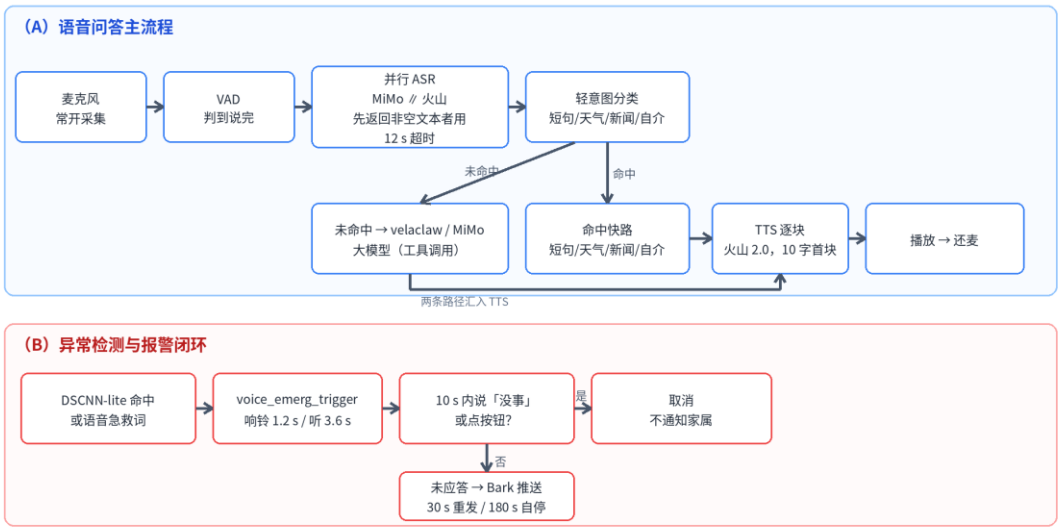


图 2 语音问答与异常报警流程

3.4.3 硬件设计与适配

本作品完成本板全套板级适配：标准 NuttX audio lower-half（录音＋播放）、显示 / 亮度 / 面板重初始化、触摸 pinmux、按键 / GPIO / 状态、RTC 日循环 alarm、USB RNDIS 上网、屏幕镜像。公共仓改动收敛为三份 working-tree 补丁，构建可复现。

是否完成全新硬件平台适配或驱动开发：是。芯片为 SF32LB52（Cortex-M33，ARMv8-M），开发板为 SF32LB52-DevKit-LCD；本作品从零完成显示、触摸、音频、USB、RTC、按键/背光与 ROMFS 等板级驱动适配，并以三份 working-tree 补丁交付。

外接模块：无，麦克风、CO5300 屏、FT6146 触摸、AW8155 功放均为板载；对外接口为 USB Type-C（RNDIS 联网 + CDC-ACM 串口 /dev/ttyACM0）与按键/背光。关键 BOM：SF32LB52（Cortex-M33）、CO5300 AMOLED、FT6146 触摸、AW8155 功放、8

MB OPI-PSRAM、16 MB NOR。

适配难点	解决方案
CO5300 面板不吃 MADCTL 换轴	横屏转置放在 LVGL flush 层完成；亮度走面板 0x51 WBRIGHT
M33 systick 配置名与上游不一致	CONFIG_ARM_M_SYSTICK → CONFIG_ARM_SYSTICK
RNDIS warm reset 后卡数据通道	烧录后拔电冷启动；补丁修复 TXPKTRDY/FIFO flush、bulk OUT 双提交与 QUERY stall
半双工音频被多 task group 争用	真实通路位 + rx_owner_gen 单读者 + write 原子串行化；报警抢音频只走 audio_play_stop
RTC read() 永远 EOF	RTC_SET_ALARM + 信号；板级 worker 设置/接收/回调，UTC+8 与日期进位手算
ROMFS 修改后不生效	删除 romfs.img、romfs_etc.c、romfs_etc/ 三个产物后重新编译
触摸 IC 偶发不应答	PC 镜像虚拟触摸兜底；报警页连续 6 秒无报警源自动撤下

驱动与适配的八个要点：

（1）芯片/架构：SF32LB52 Cortex-M33（ARMv8-M），SiFli HAL；修正 M33 systick 配置名（CONFIG_ARM_M_SYSTICK → CONFIG_ARM_SYSTICK）。

（2）内存/存储：内部 SRAM 512 KB + 8 MB OPI-PSRAM（psram @0x60000000, LENGTH=8M）；16 MB NOR，ftab@0x12000000、nuttx@0x12010000；ASR/TTS 大缓冲放 PSRAM。

（3）显示：CO5300 AMOLED 390×450，QSPI 4 线 24 MHz（全屏约 29 ms），VADD_EN PA37、复位 PA00；亮度走面板 0x51 WBRIGHT；CO5300 不吃 MADCTL 换轴，横屏只能 flush 转置。

（4）触摸：FT6146 挂 I2C0，IRQ pin 31，复位 PA09；LCD 初始化完成后再初始化触摸；触摸 IC 偶发不应答时，由 PC 镜像的虚拟触摸兜底。

（5）音频：AUDCODEC（DAC/ADC 模拟）+ AUDPRC（TX0 播放 / RX0 录音 DMA）+ AW8155 功放（PA10）；16 kHz/单声道/s16le；半双工（播放停 ADC、录音停 DAC）；录音单读者 rx_owner_gen；报警抢音频只走 audio_play_stop；write() 原子串行化，TTS 与提醒/报警不再并发写坏共享等待。

（6）USB/网络：MUSB 设备控制器；RNDIS gadget（584e:5342，全速 12 Mbps，

MAC 00:e0:4c:53:42:31）+ CDC-ACM 串口；板侧 192.168.137.2，PC 侧 192.168.137.1；补丁解决 TXPKTRDY/FIFO flush、bulk OUT 双提交、RNDIS QUERY stall；warm reset 会卡数据通道，烧录后必须拔电冷启动。

（7）RTC/报警/按键：/dev/rtc0 的 read() 永远 EOF，只能用 RTC_SET_ALARM + 信号；板级单独 worker 设 alarm、收信号、回调；未开 CONFIG_LIBC_LOCALTIME，UTC+8 与日期进位在板级手算；板级报警 worker 优先级 115，每 50 ms 写一块 PCM，alarm_trigger/clear 非阻塞。

（8）启动 /ROMFS/ 交付：board_late_initialize 初始化 RTC/ADC/audio/GPIO/PWM/buttons/I2C/SPI/LCD 异步任务/看门狗/MTD/CDC-ACM；挂载 tmpfs /data，把只读 ROMFS 的 agent_config.json 拷入；task_create 拉起 robot_ui(110)、ai_agent(100)、hello_app(100) 与 hello_app_wd(110)；改 etc/ 必须删 ROMFS 三个产物再编；上游改动以三份 working-tree 补丁交付。

3.4.4 应用/交互端设计

主屏为 390×450 圆角屏，整体采用黄白色暖调主题：奶油黄白页面底、白色圆角卡片、暖阳黄主按钮，文字为深棕/暖灰以保证对比度；顶部居中大号时钟（UTC+8），中部呼吸笑脸，底部若干大按钮。语音交互无需唤醒词：常开麦 + VAD 自动断句，说完即答。报警页为全屏红色，含大号「返回」按钮与闪烁警示图标。

移动端与网关：不依赖专用手机 App。板端通过 USB RNDIS 经 PC 侧 NAT 上网；手机侧通过 MQTT 主题 zhi_ai/zhi_ai_001/command 接收语音/报警状态并下发指令，报警走 Bark 推送（以 [PUSH] OK: HTTP 200 为判据）。PC 侧网关脚本负责 192.168.137.1/24 与 NAT 配置，串口 /dev/ttyACM0 用于日志与调试。

3.4.5 自定义 Skill

仓库内 skills/ 目录定义四个可复用能力：sound-detection（声音检测）、voice-dialog（语音对话）、emergency-alarm（急救报警）、weather-news（天气与新闻）。每个 Skill 均说明输入契约、处理流程、关键常量、相关代码与调参入口。

运行时 Skill：ai_agent 启动时从 /data/ai_agent/skills/（模板写作 /data/agent/skills/）加载 *.md 并注入系统提示。四个 Skill 的触发场景：sound-detection（非人声高能/异常声）、voice-dialog（人声/对话）、emergency-alarm（急救词/报警确认）、weather-news（天气/新闻意图）；运行时同时保留 app-launcher、device-diagnostics 等通用 Skill，至少沉淀 1 个的硬性要求已满足。

3.5 系统测试与结果分析（务必提供量化数据）

3.5.1 测试环境

项	内容
硬件	SF32LB52-DevKit-LCD（16 MB Flash / 8 MB PSRAM / 512 KB

	SRAM）
系统	openvela（NuttX）
联网	USB RNDIS，板侧 192.168.137.2 ⇄ PC 侧 192.168.137.1（NAT）
串口	/dev/ttyACM0，1000000 8N1
构建	cmake + ninja，arm-none-eabi-gcc 13.4.0
固件占用	flash 41.86%（7,022,764 B / 16 MB）；SRAM 66.75%（349,940 B / 512 KB）

3.5.2 功能测试

编号	功能	方法	结果	环境
1	语音问答	说「给我讲个小故事」	识别 → 大模型 → 火山 TTS 出声	真机
2	短句秒回	说「你好」	[短句快路]，1 秒内出声	真机
3	天气快路	说「济南天气怎么样」	[天气快路]，15~20 秒出声	真机
4	新闻快路	说「今天有什么娱乐新闻」	News via Tavily OK，念标题+摘要	真机
5	新闻概括	说「详细说说第一条」	[新闻概括] 念内容摘要	真机
6	急救关键词	喊「救命」	[急救词] 命中 → 响铃	真机
7	撞击求救	15 秒内敲两下	第一次撞击 → 第二次撞击 → 确认求救	真机
8	语音取消	铃停空档说「没事」	[急救] 老人说没事 → 停铃	真机
9	按钮取消	点「我没事,取消」	铃立刻停、页面撤下	真机
10	家属通知	报警 10 秒不吭声	[PUSH] OK: HTTP 200，30 秒重发	真机

11	自动收摊	报警满 180 秒	［急救］180 秒到点，自动收摊	真机
12	自我介绍	说「你能做什么」	［自介快路］念固定模板	真机
13	断网报警	拔 RNDIS 后喊「救命」	本地响铃仍工作	真机

3.5.3 性能测试

各环节实测耗时（多次取典型值）：

环节	实测	说明
ASR（并行赛跑）	文本到手 2~9 s（MiMo 单次 1.6~8.5 s；火山多数 -61）	MiMo 与火山同时发，12 s 超时；火山 infer≈400~500 ms
LLM（velaclaw）	7.8~10 s（简单问答）	含工具调用的题 17~25 s
TTS 首块（10 字）	2.7~3.5 s	已从 20 字降至 10 字
首次出声（普通问答）	约 20~27 s	ASR＋LLM＋TTS 首块
首次出声（天气/新闻）	15~20 s	快路，跳过 LLM
首次出声（短句）	< 1 s	预设回复，无网络
端侧推理	5 窗/s（200 ms/窗），单窗预算 200 ms；2372 参数常开	未单独打点前向 ms；门控线程低优先级，不占 PSRAM
UI 刷新	flush 4~8 次/s，最慢 flush 10 ms；lv_timer_handler 最慢 60~90 ms	390×450 黄白暖调界面
资源占用	Flash 7.13 MB / 42.5%；SRAM ≈350 KB / 512 KB；模型 9.5 KB	8 MB PSRAM 用于 ASR/TTS 大缓冲
功耗	未测（原型阶段，无功率计）	如实标注，不作为已达成指标

结论：ASR 与 LLM 属云端推理时间，本地不可压缩；本版优化集中在 TTS 首块与意图快路，并将长回复的 TTS 分块上限由 64 字降回 32 字，解决了长文本「说到一半停止」的问题。

3.5.4 可靠性与稳定性测试

本版针对真机发现的多类整机级不稳定做了根因修复：

问题	真机指纹	根因	修复
UI 线程冻结	[ui] 慢统计 停止 / 按钮无响应	pthread_mutex 持有者跟踪 + audio_play_stop 阻塞界面	全部改二值信号量；取消时 detached 线程掐铃
录音卡读 8 秒	录音线程卡在设备读里 8019 ms / 放弃 join	旧单读者判据放行新读者、取消旧读者心跳	rx_owner_gen 单读者守卫
断言打死线程	Assertion failed at sem_post.c:63	内核 mutex 持有者字段被破坏	audio_in_lock 改二值信号量
TTS 说到一半停	火山 2.0 合成失败(-22) → 回落 MiMo → TTS 停滞 25 s 作废	分块 64 字超出火山接受范围	分块降回 32 字
识别成功却无回复	识别结果迟到（快照第 5 轮，现在第 6 轮），作废	录音卡读后轮次跳变	差 1 轮也复用结果
报警语音「没事」无效	[volc_asr] No text recognized；听窗峰值仅 444	ASR 音频过轻；提示音抢麦；短句判定按字节	补静音+最多 6× 增益（全对话生效）；报警中禁播提示音；按 UTF-8 字符数判定
重置后语音永久失效	[轮次] 轮 N 卡在 ASR 相位 30006 ms，作废	旧识别线程卡在 ASR，新实例误判“线程已在跑”而沿用	识别线程加入实例代际；新实例必建新线程，旧线程醒来即退出
天气 / 检	Open-Meteo 两次	网络段仍按	发起网络前

问题	真机指纹	根因	修复
索拖垮轮次	TLS 约 27 s 后轮次作废	ASR 相位 30 s 计时	切到 LLM 相位（45 s）
TCP 连接只涨不降	[TCP] pool 6→29 st[D21 …]，最终 socket ENOMEM	close work 先置 DIRECT，FIN_WAIT_2 时跳过 free 且无人再回调；TIME_WAIT 到期只置 CLOSED 不释放	TIME_WAIT 到期补 tcp_free；tcp_alloc 主动回收僵尸 CLOSED；45 分钟 pool 5~7/D0~1
报警页僵尸 / 触摸失效	UI/MQTT 正常但按钮无响应	触摸 IC 不应答 + 页面无自动收摊	报警源结束后 6 秒自动撤页；报警仍响时不动

量化指标：端侧窗口准确率 80.6%（离线验证），负样本窗口误报率 6.8%；漏报率未单独统计，采用 3/4 窗口投票、单次置信度 ≥0.80 或 15 秒内两次撞击、语音急救词与报警二次确认多路径兜底。

连续运行与资源波动：已完成 45 分钟 TCP 连接池回归（pool 5~7 / D 0~1）与多轮真机对话回归；SRAM 约 350 KB / 512 KB 基本持平，UI flush 4~8 次/s、慢 flush 0；>2 小时长时间老化仍待复测。

异常恢复：监听守护检测到「要常听却没在录」会停掉僵尸会话并重开；UI 看门狗 10 秒无心跳打印调用栈；报警页在报警源结束后 6 秒自动撤下；ASR 结果迟到 1 轮仍复用；TCP 僵尸条目主动回收；长连接由 keep-alive 探测半开连接。

未闭环项（如实记录）：TLS 出站连接泄漏已从内核根因修复（TIME_WAIT 到期补 tcp_free + 主动回收僵尸 CLOSED 条目），已完成 45 分钟真机回归（pool 5~7 / D 0~1），更长老化仍需复测；录音链路在极端负载下的 HAL 读等待仍偶发 8 秒卡读，监听守护会重开会话，根因与写路径同类，仍需继续收口。

3.5.5 稳定性收口与语音链路优化

以下修复均以真机串口日志定位，代码与补丁同步在 官方仓库及 patches/*-working-tree.patch。

（1）报警语音取消「没事」：听窗内「没事」原始峰值仅 444（正常说话 3000+），火山判“无人声”；且报警期间 AI 仍播「没听清」提示音，半双工抢走麦克风。修复为：追问/急救短句补静音到 1 秒、峰值低于 1500 时最多 8× 增益；报警期间一律不播「没听清/识别失败」；短句判定由 strlen 字节数改为 UTF-8 字符

数（「我没事，取消报警」= 8 字，不再被当成 24 字节长句）。真机日志：追问短句增益 x5（原始峰值 1064）→ 老人说没事 -> 停铃。该低电平增益随后扩展到所有对话音频（峰值低于 3000 时最多 6×），真机 x2/x6 片段均恢复识别。

（2）重置后识别线程僵死：上一代识别线程卡在 ASR 网络调用里，static 的“线程已在跑”标记仍为真，新实例不再建线程，语音请求投入无人处理的信号量，30 秒后轮次作废。修复：每次初始化递增实例代际，工作线程只服务自己那一代；新实例发现代对不上必建新线程；旧线程醒来立即退出且不碰当前代标记。

（3）快路网络段误触发轮次作废：天气/实时检索在发起 TLS 时轮次仍处于 ASR 相位（30 秒上限），Open-Meteo 两次握手约 27 秒即把轮次作废并连累后续轮次。修复：网络请求前切换到 LLM 相位（45 秒）。真机验证连问两次天气均正常回答，0 次“卡在 ASR 相位”。

（4）TCP 连接只涨不降（内核）：tcp_close_work 先置 TCP_CLOSE_DIRECT，执行时状态为 FIN_WAIT_2，跳过 tcp_free 后无人再回调；2MSL 后 tcp_timer 只把状态置 CLOSED，close 回调已释放，条目永久占用 mempool。3 分钟真机 nalloc 6→29、21 条 CLOSED。修复：TIME_WAIT 到期且 crefs==0、DIRECT 时直接 tcp_free；池满时允许回收已跑过 close work 的 CLOSED 条目。 此外在 tcp_alloc 中每新建一条连接主动回收一条 crefs==0、CLOSED、close work 已跑过的僵尸条目（服务端 RST/短连接会绕过 TIME_WAIT）。真机 45 分钟回归：pool 稳定 5~7、D 0~1；修复前同长度测试为 pool 6→29、D0→21。

（5）实时信息简化与交互补全：删除娱乐/体育/财经分类与「第 X 条」专门路径，改为用户原话直接交 Tavily 通用检索；输出缓冲 1024→12288 字节（旧版 1023 字节把 10 条截成 1 条）；过滤版权/页脚行。语音聊天页右上角新增 56×40「返回」按钮；UI 看门狗心跳改用单调时钟。

（6）僵尸报警页自动收摊：触摸 IC 偶发不应答时，红色报警页会挡在主界面上面，人点不了「返回」；但 UI 线程、MQTT 心跳均正常，stop_alarm 也能收到。UI 主循环每约 1 秒检查：报警页在最上层、alarm 未激活、voice_emerg 未激活，连续 6 秒则自动撤页并撤掉残留询问页；报警仍在响时不动。

（7）本地时间快路：问「现在几点/今天几号/星期几」不再走云端 LLM，直接读 RTC（UTC+8）组句交给 TTS。真机 ASR 6.9 s + 本地回复 10 ms + TTS 2.2 s，首响约 9 秒；省掉的是 LLM 的 5~11.5 秒。

（8）简单问答直连 MiMo（已关闭）：真机实测直连 LLM 段为 11.9 s / 10.5 s / 26.3 s，反而比 velaclaw 更慢，且占住唯一识别线程，导致后续轮次 30 秒看门狗作废（用户侧表现为「有的甚至不回复」）。已将 AI_SIMPLE_CHAT_DIRECT 置 0，问答统一回 velaclaw，提醒/音量/天气/搜索等工具链路保持不变；代码保留，网关稳定后可通过宏重新启用。

（9）TCP 连接池主动回收：在 TIME_WAIT 到期补 tcp_free 的基础上，tcp_alloc 每新建一条连接主动回收一条 crefs==0、CLOSED、close work 已跑过的僵尸条目；真机 45 分钟回归 pool 稳定 5~7、D 0~1。

（10）播放看门狗误杀 TTS 尾音：真机一条回复的每一块 TTS 在 5 秒后出现

「播放卡死：阶段=已收尾」；播放线程已把整块写进设备并走完 close，DMA 缓冲里还有尾音，play_progress_ms 只在 write 时更新，5 秒无新写入被误判为停滞并强制收摊。修复：stage==DONE 时跳过停滞判据；CLOSE/DONE 诊断标记只允许当前代线程写。判据：长故事不再出现该行，能一块接一块念完。

（11）TTS 与提醒/报警并发写破坏共享等待：TTS（hello_app）与提醒铃/报警（robot_ui）是两个 task group，共用驱动里的 wr_sem / wr_wait_wdog；第二个 write 的 nxsem_reset + wd_cancel 会拆掉第一个正在等的超时，使第一个永久卡在 write。修复：write() 入口用原子占位串行化，同一时刻只允许一个 write 进设备；第二个最多等 2 秒，等不到返回 -EBUSY 丢块，绝不再 reset/cancel 别人的等待状态。

（12）低电平 ASR 增益：真机多段语音峰值仅 200~800（正常 3000+），火山返回 -61；追问/急救短句补静音到 1 秒，普通对话在峰值低于 3000 时增益到约 8000（上限 10×），只改送 ASR 的副本。报警听窗火山空结果时用 MiMo 二次识别，只采纳包含「没事/不用/没有」的结果，命中即取消，避免幻觉。

（13）ASR 双后端并行赛跑：MiMo 线程先创建、火山线程同时创建，谁先返回非空文本就用谁；12 秒超时；PCM 引用计数保证 loser 安全读完；任一端不可用退回串行识别。真机 25 轮：MiMo 25 次出字，火山 24 次返回、多数为 -61（ENODATA，未听出字）；火山唯一出字一轮 3830 ms，略快于同轮 MiMo 的 4122 ms。

（14）TCP keep-alive：vela_tls.c 在每条 TLS socket 上设置 SO_KEEPALIVE，板级 defconfig 打开 CONFIG_NET_TCP_KEEPALIVE=y；连接池长连接对端悄悄消失时由内核探测并标记，减少复用半开连接等到超时。串口判据：[vela_tls] keepalive on fd=N（真机 92 次）。

3.6 AI-Native 开发说明

指标	数据
AI Coding 代码占比	约 99%（AI 生成/改写代码行占比；人工负责架构、调试与验收）
使用的 AI 工具	Claude Code、MiMo Agent、deepseek、kimicode
MCP 工具使用情况	未接入外部 MCP；使用 Claude Code 内置文件/终端/搜索工具与串口日志采集脚本
Skills 使用与新增情况	使用并沉淀 4 个：sound-detection、voice-dialog、emergency-alarm、weather-news

Token 使用总量	3000000000 左右

本项目坚持「先证据、后改动」：每一项修复都对应串口日志中的唯一指纹（见 docs/TROUBLESHOOTING.md），没有日志证据的猜测不改代码。AI Coding 对话日志按成员与日期保存在 logs/，随提交材料一并提交；仓库内 skills/ 目录沉淀了四个可复用的开发与维护 Skill。

3.7 总结与展望

3.7.1 成果总结

（1）完成本板全套板级适配

标准 NuttXaudiolower-half（录音+播放）、显示/亮度/面板重初始化、触摸 pinmux、按键/GPIO/状态、RTC 日循环 alarm、USBRNDIS 上网、屏幕镜像；公共仓改动收敛为三份补丁，构建可复现。

（2）端侧 AI 从零做到可常开

手写512点radix-2FFT的log-mel前端+2372参数DSCNN-lite，3/4投票后处理，零外部依赖；PC 侧训练、导出、对拍脚本随仓库交付。

（3）端侧与云端分流机制落地

人声门控把"说话"交给云端、"异常声"交给本地小模型；播放期间整窗跳过，从机制上消掉了"自家 TTS 被当成跌倒"这类误报。

（4）主动守护链路闭环

本地小模型 / 语音关键词命中 → 响铃 → 确认 → 未应答则 Bark 通知 → 30 秒重发 → 180 秒自停；说「没事」、点按钮、超时多条路径收口到同一取消入口。

（5）高频意图快路

天气 / 新闻 / 自我介绍 / 短句不经过大模型，首次出声由约 105 秒降至 15~20 秒，短句低于 1 秒。

（6）可靠性地基

全部互斥锁改二值信号量消除 UI 死锁；录音单读者 rx_owner_gen 消除卡读；TTS 分块与降级策略调整消除整机无声；UI 看门狗提供卡死诊断能力。

（7）文档与交付

README ＋ 6 份系统文档 ＋ 3 份权威补丁 ＋ 4 个 Skill ＋ AI Coding 日志。

3.7.2 应用前景与商业价值

（1）目标受众：65 岁以上独居/空巢老人及其异地子女；地域上以城市社区居家养老为主。使用偏好是"零学习成本"：不用操作 App，说话就行，按钮只用大字号几个。

（2）场景落点：居家安全看护（异常声响 / 主动提醒 / 一键报警）＋情感陪伴（语音聊天）。

（3）商业模式：硬件整机一次性+云端服务订阅制（ASR/LLM/TTS 是持续成本）；面向养老机构 / 社区照护平台是更低获客成本的第二条路径。

（4）规模化潜力与关键前提：硬件是低 BOM 的 MCU 方案（本板只要 9.5KB），真正门槛在云端 token 费用与误报控制。本作品当前为技术原型，无成本、用户试用与规模数据，不给出收入 / 规模预测。

3.7.3 不足与未来工作

（1）训练数据严重不平衡：fall / knock 样本量远小于 other，弱势类召回率偏低，需在板端补录。

（2）弱音/噪声下的 ASR：火山可能返回 -61（ENODATA），MiMo 在低信噪比时会编字（实测「Okay.」「Yeah.」「八百万。」）。本版用并行赛跑降低等待，但结果采纳仍需按音频能量/置信度门控；下一步引入流式 ASR 与门控策略。

（3）普通问答首次出声仍约 15~25 秒：ASR 文本到手 2~9 秒、velaclaw 5~11.5 秒、TTS 首块 2~3 秒；进一步压缩需流式 ASR 与 token 级流式 LLM。

（4）实时信息条数受限：中文实时检索在 Tavily 索引中约 1~3 条；已取消「第 X 条详细说说」专门路径，追问交给大模型。

（5）TLS 连接池泄漏：已从内核根因修复（tcp_timer/tcp_conn），45 分钟回归 pool 5~7 / D 0~1；更长老化仍待复测。

（6）持久化：/data 为 tmpfs，提醒、设置、模板断电丢失；产品化需接 littlefs/MTD 持久分区。

（7）本板无 IMU 与摄像头：姿态跌倒检测、视觉识别需更换硬件后再议。

（8）功耗未测：原型阶段无功率计，后续需补功耗与温升数据。

（9）天气配置：和风天气为可选增强，未配置时自动走 Open-Meteo 免 key 兜底，城市精度略降。